Shunyaya Structural Computation Core (SSC-Core)

Deterministic Structural Work Geometry & Execution Topology Framework

---

Version: Public Open Standard (v1.4)
Release Date: March 5, 2026
Status: Public Open Standard (Deterministic Structural Computation Layer)
License: Open Standard — specification may be implemented freely; provided "as is" without warranty or liability.
Caution: Deterministic structural work measurement and replay verification only.

SSC-Core is:

- Not a predictive engine
- Not an optimization system
- Not a probabilistic evaluator
- Not a cryptographic validator
- Not an AI scoring framework
- Not a safety-critical monitoring layer

Scope Notice

Shunyaya Structural Computation Core (SSC-Core) defines a **deterministic structural computation framework** that measures execution work geometry alongside classical output generation.

SSC-Core operates as a **structural computation layer** that extends runtime execution with measurable work topology — without altering output truth.

SSC-Core:

- Does **not** modify operator definitions
- Does **not** alter numerical magnitude
- Does **not** redefine correctness
- Does **not** inject probabilistic inference
- Does **not** approximate outputs
- Does **not** optimize by changing answers
- Does **not** simulate physics
- Does **not** predict outcomes
- Does **not** evaluate semantic validity of outputs

SSC-Core performs one function:

It computes structural work geometry under deterministic replay discipline.

SSC-Core converts any deterministic execution into a **structural computation topology** through:

- Structural work field computation
- Compression integral derivation
- Curvature quantification
- Deterministic topology fingerprinting
- Replay-verifiable certificate generation

Conforming implementations shall publish:

- Declared operator definition
- Domain definition
- Work model definition `W(t)`
- Allow-rule definition for `W_allow(t)`
- Structural work field `Psi(t)`
- Compression integral `Omega`
- Structural curvature `K`
- Topology fingerprint `F`
- Deterministic trace artifacts (CSV outputs + SHA-256 manifest)
- Exact reproduction commands

Sufficient documentation must be provided to enable **independent deterministic replay**.

Replay determinism is the conformance standard.

Authority condition:

```
B_A = B_B
```

---

0. Abstract

Shunyaya Structural Computation Core (SSC-Core) is a deterministic structural computation framework for **execution geometry measurement**.

Given a deterministic operator that produces a classical output `m_out`, SSC-Core computes a parallel structural work geometry layer while preserving the classical result exactly.

SSC-Core introduces measurable execution geometry without altering the underlying computation.

The collapse invariant must always hold:

```
phi((m,a,s)) = m
```

Where:

`m` = classical magnitude
`a` = structural alignment state
`s` = structural state

Magnitude remains unchanged.

SSC-Core therefore observes execution structure while preserving classical output truth.

Given a deterministic operator that produces a classical output `m_out`, SSC-Core computes a parallel structural work geometry layer consisting of:

- Structural work field `Psi(t)`
- Structural compression integral `Omega`
- Structural curvature `K`
- Deterministic topology fingerprint `F`

Core structural quantities:

- `Psi(t) = W_allow(t) / W(t)`
- `Omega = integral_0^T (1 - Psi(t)) dt`
- `K = sum_{i=2..N} |g(i) - g(i-1)|`
- `F = hash(g_stream || "|" || Psi_samples || "|" || Omega || "|" || K)`

Where:

- `W(t)` = cumulative classical work
- `W_allow(t)` = cumulative structurally executed work
- `g(i)` = deterministic execution decision stream

SSC-Core introduces:

- Deterministic structural work field computation
- Compression-area measurement
- Execution curvature topology
- Structural execution fingerprinting
- Trace-derived certificate enforcement
- Replay-verifiable evidence discipline

All structural quantities are:

• deterministic
• trace-derived
• replay-verifiable
• manifest-certifiable

SSC-Core does not measure randomness.
SSC-Core does not compute entropy.
SSC-Core does not estimate probability distributions.
SSC-Core does not alter classical outputs.

Instead, SSC-Core answers a structural question:

What is the measurable geometry of work that produced this output?

This establishes a new deterministic axis for:

• algorithm comparison
• execution topology classification
• structural compression analysis
• deterministic performance geometry
• cross-implementation structural fingerprinting
• replay identity certification

Two executions may produce identical classical outputs while exhibiting different structural execution geometry.

SSC-Core makes these differences measurable without altering the underlying computation.

SSC-Core preserves classical magnitude exactly.

Collapse invariant:

```
phi((m,a,s)) = m
```

SSC-Core therefore operates purely at the level of structural execution geometry.

TABLE OF CONTENTS

Shunyaya Structural Computation Core (SSC-Core) 1
0. Abstract 2
1. Foundational Invariants & Output Preservation Discipline 5
2. Replay Identity & Certificate Architecture 10
3. Structural Computation Model & Formal Geometry of Work 16
4. Conformance Specification & Prohibited Behaviors 22
5. Structural Applications, Boundaries & Non-Claims 27
Appendix A — Mathematical Foundations of Structural Work Geometry 32

---

1. Foundational Invariants & Output Preservation Discipline

1.1 Conservative Output Preservation

SSC-Core is a **conservative structural overlay**.

It computes additional structural quantities **without altering classical outputs**.

Non-negotiable collapse invariant:

```
phi((m,a,s)) = m
```

Where:

- `m` = classical magnitude or output
- `a` = structural alignment state
- `s` = structural posture state

This invariant guarantees:

- Output truth is preserved exactly
- Structural computation does not override classical correctness
- No smoothing, reinterpretation, or proxy substitution occurs
- No structural computation replaces domain mathematics

SSC-Core may compute additional quantities — but it **must never modify `m_out`**.

1.2 Deterministic Extension of Execution State

SSC-Core extends the runtime state with a structural work axis.

Extended computational state:

```
X(t) = (m(t), a(t), s(t), w(t))
```

Where:

- `m(t)` = classical magnitude stream
- `a(t)` = structural alignment posture
- `s(t)` = structural state
- `w(t)` = cumulative structural work accounting state

The introduction of `w(t)` creates a **computable structural dimension** alongside classical execution.

This is not logging.
This is not metadata annotation.

It is a deterministic computational extension.

1.3 Deterministic Work Accounting Requirement

SSC-Core requires explicit declaration of the work model.

Let:

- `W(t)` = cumulative classical work up to time `t`
- `W_allow(t)` = cumulative structurally executed work up to time `t`

Both must be:

- Deterministically defined
- Fully declared in operator identity record
- Replay-reproducible
- Independent of runtime adaptation

No hidden counters permitted.
No probabilistic accounting permitted.
No tolerance-based estimation permitted.

Work discipline must be explicit and reproducible.

1.4 Structural Work Field Definition

SSC-Core computes the **Structural Work Field**:

```
Psi(t) = W_allow(t) / W(t)
```

Properties:

- `0 < Psi(t) <= 1`
- Deterministic
- Monotonic under cumulative work discipline
- Replay-stable

Interpretation:

- `Psi(t) = 1` → fully classical execution
- `Psi(t) < 1` → structurally compressed execution

`Psi(t)` is not a performance metric.

It is a **structural execution field intrinsic to computation itself**.

Classical computation has no equivalent quantity.

1.5 Structural Compression Integral

SSC-Core computes structural compression as a deterministic integral.

```
Omega = integral_0^T (1 - Psi(t)) dt
```

Discrete execution equivalent:

```
Omega = sum_{i=1..N} (1 - Psi(i))
```

Properties:

- `Omega >= 0`
- `Omega = 0` under full classical collapse mode
- Deterministically derivable from trace
- Replay-identical under identical execution

`Omega` measures the **area removed from classical work**.

It is:

- Computed
- Not interpreted
- Not estimated
- Not probabilistic

1.6 Structural Curvature of Execution

Let the deterministic execution decision stream be:

`g(i) = 1` if work executed at step `i`
`g(i) = 0` if work structurally skipped at step `i`

Structural curvature:

```
K = sum_{i=2..N} |g(i) - g(i-1)|
```

Properties:

- `K >= 0`
- `K = 0` → structurally smooth execution
- Higher `K` → structural oscillation or turbulence
- Deterministically computable from trace

`K` is a topological property of the execution path.

Classical output alone cannot reveal `K`.

1.7 Structural Topology Fingerprint

SSC-Core computes a deterministic fingerprint of structural execution geometry.

Canonical form:

```
F = hash(g_stream || "|" || Psi_samples || "|" || Omega || "|" || K)
```

Where:

- `g_stream` = full deterministic decision sequence
- `Psi_samples` = fixed deterministic sampling schedule
- Concatenation format is fixed and declared
- Hash function is deterministic and declared

Properties:

- Replay-identical under identical execution
- Sensitive to any structural modification
- Trace-derivable
- Certificate-verifiable

Two executions may produce identical `m_out`.

Only SSC-Core determines whether their structural geometry is identical.

Structural identity condition:

```
F_A = F_B
```

Replay authority condition:

```
B_A = B_B
```

1.8 Determinism Requirement

All SSC-Core structural quantities must be:

- Deterministic
- Trace-derived
- Recomputable from artifacts
- Byte-identical under replay

Forbidden:

- Randomness
- Probabilistic thresholds
- Adaptive horizon resizing
- Tolerance-based equality
- Floating approximation drift
- Runtime parameter mutation

If structural quantities cannot be recomputed exactly from trace artifacts, the implementation does not conform.

1.9 Collapse Mode Discipline

SSC-Core must support a deterministic collapse mode.

Collapse mode definition:

- `W_allow(t) = W(t)` for all `t`
- `Psi(t) = 1` for all `t`
- `Omega = 0`
- `K = 0` (or minimal under declared decision encoding)

Collapse mode guarantees:

- Full classical equivalence
- Structural measurement without compression
- Deterministic baseline for comparison

Collapse mode is mandatory for civilization-grade conformance.

---

2. Replay Identity & Certificate Architecture

2.1 Replay Identity — Authority Condition

SSC-Core is civilization-grade only if execution identity is provable.

Authority condition:

```
B_A = B_B
```

Where:

- `B_A` = manifest of primary execution
- `B_B` = manifest of replay execution

Equality requires:

- Byte-identical artifacts
- Identical structural quantities
- Identical topology fingerprint `F`
- Identical trace stream
- Identical manifest hash

There is:

- No tolerance
- No approximate equality
- No statistical equivalence
- No interpretation-based acceptance

If `B_A != B_B`, conformance fails.

Replay identity is the highest authority.

2.2 Deterministic Artifact Requirements

Each SSC-Core execution must produce a deterministic artifact capsule.

Required Core Artifacts

- `ssc_trace.csv`
- `ssc_summary.txt`
- `RUN_CONFIG.txt`
- `RUN.txt`
- `MANIFEST.sha256`

These artifacts must be:

- Deterministically generated
- Fully reproducible
- Environment-normalized
- Byte-stable across independent runs

No nondeterministic ordering permitted.
No timestamp injection permitted.
No runtime randomness permitted.

2.3 Operator Identity Record (OIR)

Each implementation must publish an **Operator Identity Record** declaring:

- Operator name
- Domain definition
- Input identity rule
- Work model rule (`W(t)` definition)
- Allow rule (`W_allow(t)` definition)
- Structural decision encoding (`g(i)` definition)
- Sampling schedule for `Psi(t)`
- Fingerprint construction rule for `F`
- Hash algorithm used

OIR must be:

- Explicit
- Complete
- Deterministic
- Sufficient for independent reproduction

If OIR is incomplete, replay conformance cannot be validated.

2.4 Structural Certificate Requirements

Each SSC-Core run must generate a **Structural Certificate** containing:

- Input digest
- Output digest
- Total classical work `W`
- Total allowed work `W_allow`
- Endpoint `Psi(T)`
- Sampling rule for `Psi(t)`
- Compression integral `Omega`
- Structural curvature `K`
- Topology fingerprint `F`
- Replay identity status
- Final PASS / FAIL status

The certificate must be:

- Derivable from trace artifacts
- Recomputable independently
- Deterministically verifiable

Certificates must never rely on narrative assertions.

All declared values must be trace-derivable.

2.5 Trace-Derived Enforcement Rule

Structural quantities must be recomputed from:

`ssc_trace.csv`

Independent recomputation must produce:

- Identical `Omega`
- Identical `K`
- Identical `F`
- Identical `Psi` summary

If recomputed values differ, certificate fails.

Structural authority flows from trace — not from printed summary.

2.6 Manifest Discipline

Each artifact bundle must include:

```
MANIFEST.sha256
```

The manifest must:

- Include all structural artifacts
- Use deterministic ordering
- Be generated after artifact freeze
- Be replay-identical

The manifest defines the authoritative identity of the execution.

Replay identity condition:

```
hash(MANIFEST_A) = hash(MANIFEST_B)
```

2.7 Structural Freeze Boundary

A certified SSC-Core release must define a deterministic freeze boundary including:

- `scripts/`
- `reference_outputs/`
- `release/` (if applicable)
- Verification utilities

Excluded from freeze boundary:

- `docs/`
- README files
- Diagrams
- Visualization tools
- Explanatory materials

Modification inside the frozen boundary invalidates certification.

Presentation materials may evolve independently.

Structural core must remain immutable once certified.

2.8 Multi-Stage Verification Discipline

Civilization-grade verification must include:

Stage 1 — Primary Execution

Generate full artifact bundle.

Stage 2 — Replay Execution

Regenerate artifacts independently.

Stage 3 — Replay Comparison

Validate:

```
B_A = B_B
```

Stage 4 — Trace Re-Derivation

Recompute structural quantities from trace.

Stage 5 — Manifest Equality

Validate manifest hash equality.

All stages must PASS.

Any failure invalidates conformance.

2.9 Tamper Ritual (Integrity Enforcement)

Civilization-grade systems must enforce deterministic tamper detection.

Required ritual:

1. Execute primary run
2. Execute replay run
3. Confirm `B_A = B_B`
4. Modify one byte in any artifact
5. Re-run verification → must FAIL
6. Restore artifact → must PASS

No ambiguous outcomes permitted.

If tampering does not trigger failure, the implementation does not conform.

2.10 Deterministic Environment Requirement

Implementations must ensure strict environmental determinism.

The execution environment must guarantee:

- Stable file ordering
- Explicit encoding rules (character encoding must be declared and fixed)
- Deterministic floating-point arithmetic handling
- Explicit and fixed rounding discipline
- Fixed hash algorithm (declared in OIR)
- Fixed output formatting rules
- Platform-independent byte serialization format

Floating-point computations must be explicitly normalized to deterministic precision rules.

All numeric formatting, rounding, and serialization must be:

- Fully specified
- Platform-independent
- Encoding-stable
- Replay-identical

Execution must not depend on:

- System clock
- Random seed
- Thread scheduling
- Parallel nondeterminism
- Unordered container iteration
- OS-dependent file system ordering
- Locale-dependent formatting

If identical declared inputs executed under different environments produce artifact differences, conformance fails.

Environmental nondeterminism invalidates replay identity.

Final authority condition:

```
B_A = B_B
```

Strict determinism is mandatory.

2.11 Civilization-Grade Condition

SSC-Core is civilization-grade only if:

- `phi((m,a,s)) = m` preserved
- Structural quantities are trace-derivable
- Replay identity enforced
- Freeze boundary declared
- Tamper ritual passes
- No nondeterminism present

Final authority condition:

```
B_A = B_B
```

Execution identity is the standard.

---

3. Structural Computation Model & Formal Geometry of Work

3.1 Classical Execution Model

Let a deterministic operator produce an output `m_out` over discrete steps `1..N`.

Classical computation defines:

• a state evolution
• a final output
• a cumulative work measure

Let cumulative classical work be:

`W(t)` for `t = 1..N`

with:

```
W(N) = W
```

Classical execution stops at output.

It does not measure the **geometry of work**.

In SSC-Core instrumentation, deterministic execution traces expose the cumulative classical work stream `W(t)` and the structurally executed work stream `W_allow(t)`.

These traces are emitted as deterministic step records in `ssc_trace.csv`, enabling structural quantities such as `Psi(t)`, `Omega`, `K`, and `F` to be computed directly from execution evidence.

The classical operator itself remains unchanged.

SSC-Core therefore observes execution structure **without altering the classical computation model**.

3.2 Structural Execution Model

SSC-Core extends classical execution by defining a structural execution decision stream:

```
g(i) in {0,1}
```

Where:

- `g(i) = 1` → work executed at step `i`
- `g(i) = 0` → work structurally skipped at step `i`

Allowed cumulative work:

```
W_allow(t) = sum_{i=1..t} g(i) * w_i
```

Where:

- `w_i` = declared classical work increment at step `i`

Structural discipline requires:

- `0 <= W_allow(t) <= W(t)`
- Deterministic definition of `w_i`
- Deterministic definition of `g(i)`

No adaptive runtime mutation permitted.

3.3 Structural Work Field

SSC-Core defines a runtime structural field:

```
Psi(t) = W_allow(t) / W(t)
```

Properties:

- `0 < Psi(t) <= 1`
- Deterministic
- Replay-stable
- Monotonic if cumulative work discipline enforced

`Psi(t)` transforms execution into a measurable geometric field.

Classical execution has no such field.

3.4 Discrete Structural Compression

Discrete compression:

```
Omega = sum_{i=1..N} (1 - Psi(i))
```

Continuous analog:

```
Omega = integral_0^T (1 - Psi(t)) dt
```

Properties:

- `Omega >= 0`
- `Omega = 0` under full collapse mode
- Deterministically derivable
- Independent of output magnitude

`Omega` measures structural area removed from classical work.

It is a geometric quantity.

3.5 Structural Curvature

Structural curvature:

```
K = sum_{i=2..N} |g(i) - g(i-1)|
```

Interpretation:

- `K = 0` → smooth execution topology
- Larger `K` → structural oscillation
- `K` is independent of output value
- `K` is purely topological

`K` captures structural turbulence in the execution path.

Two executions with identical output may have different `K`.

3.6 Structural Density

Define structural density:

```
D = W_allow(N) / W(N)
```

Properties:

- `0 < D <= 1`
- `D = 1` → fully classical execution
- Lower `D` → greater structural compression

Density relates global compression to total work.

`D` and `Omega` measure different geometric aspects:

- `D` = endpoint compression ratio
- `Omega` = cumulative compression area

3.7 Execution Geometry Vector

SSC-Core defines a structural execution geometry vector:

```
G = (Omega, K, D, F)
```

Where:

- `Omega` = compression area
- `K` = curvature
- `D` = structural density
- `F` = topology fingerprint

This vector characterizes execution geometry independently of output.

Two executions are structurally identical if and only if:

- `Omega_A = Omega_B`
- `K_A = K_B`
- `D_A = D_B`
- `F_A = F_B`

Replay identity still requires:

```
B_A = B_B
```

3.8 Structural Equivalence Classes

SSC-Core introduces structural equivalence classes:

Two executions belong to the same structural class if:

- They produce identical output
- They produce identical structural geometry vector `G`

Formally:

```
(m_out_A = m_out_B) AND (G_A = G_B)
```

Classical computation only compares outputs.

SSC-Core compares output and geometry.

3.9 Structural Divergence

Structural divergence occurs when:

```
m_out_A = m_out_B
```
but
```
G_A != G_B
```

This reveals:

- Hidden compression differences
- Topological differences
- Execution path instability
- Structural non-equivalence

SSC-Core makes structural divergence measurable.

3.10 Collapse Baseline

Collapse mode enforces:

- `g(i) = 1` for all `i`
- `W_allow(t) = W(t)`
- `Psi(t) = 1`
- `Omega = 0`
- `K = 0`
- `D = 1`

Collapse mode establishes:

- Deterministic geometric baseline
- Classical equivalence reference
- Structural comparison anchor

All structural compression must be measurable relative to collapse mode.

3.11 Structural Fingerprint Canonicalization

Fingerprint construction must be:

- Fully declared
- Deterministic
- Order-stable
- Encoding-stable

Canonical example:

```
F = hash(g_stream || "|" || Psi_samples || "|" || Omega || "|" || K)
```

Properties:

- Sensitive to any structural mutation
- Stable under replay
- Deterministically reconstructible

Fingerprinting elevates geometry to identity.

3.12 Formal Structural Identity

Complete structural identity requires:

- `phi((m,a,s)) = m`
- Deterministic `Psi(t)`
- Deterministic `Omega`
- Deterministic `K`
- Deterministic `F`
- Replay identity `B_A = B_B`

Structural identity is not interpretive.

It is computed.

---

4. Conformance Specification & Prohibited Behaviors

4.1 Binary Conformance Rule

SSC-Core conformance is binary.

An implementation either:

- Satisfies all deterministic structural requirements
- Or does not conform

There is:

- No partial conformance
- No approximate conformance
- No SSC-inspired category
- No interpretive equivalence

Conformance authority is defined solely by replay identity:

```
B_A = B_B
```

If replay identity fails, conformance fails.

4.2 Mandatory Structural Requirements

A conforming implementation must:

1. Preserve classical magnitude exactly:

   ```
   phi((m,a,s)) = m
   ```

2. Declare deterministic work model `W(t)`
3. Declare deterministic allow rule `W_allow(t)`
4. Declare deterministic execution decision stream `g(i)`
5. Compute structural work field:

   ```
   Psi(t) = W_allow(t) / W(t)
   ```

6. Compute structural compression:

   ```
   Omega = sum_{i=1..N} (1 - Psi(i))
   ```

7. Compute structural curvature:

   ```
   K = sum_{i=2..N} |g(i) - g(i-1)|
   ```

8. Compute deterministic topology fingerprint `F`

9. Produce deterministic trace artifacts
10. Produce manifest hash
11. Support deterministic replay verification

All structural quantities must be:

- Deterministic
- Trace-derivable
- Replay-stable
- Byte-identical under identical inputs

4.3 Hyperparameter Lock Requirement

All declared parameters must remain fixed during execution.

The declared configuration must include at minimum:

- Operator identity
- Domain size `N`
- Work model rule
- Allow rule
- Sampling schedule
- Fingerprint construction rule
- Hash algorithm

No runtime adaptation permitted.

If configuration changes during execution, conformance fails.

Hyperparameter mutation invalidates replay identity:

```
B_A = B_B
```

4.4 Deterministic Replay Requirement

Two independent executions under identical declared inputs must produce:

- Identical trace files
- Identical structural quantities
- Identical fingerprints
- Identical manifests

Replay identity condition:

```
B_A = B_B
```

Equality requires:

- Byte-identical artifacts
- Identical encoding
- Identical ordering

There is:

- No tolerance
- No approximate equality
- No floating epsilon comparison
- No statistical similarity

Replay identity is mandatory.

4.5 Prohibited Behaviors

An implementation does not conform if it introduces:

- Randomness
- Probabilistic inference
- Adaptive thresholds
- Runtime parameter mutation
- Floating tolerance acceptance
- Timestamp injection into artifacts
- Nondeterministic output ordering
- Parallel nondeterministic reduction
- Heuristic smoothing
- Approximate hashing
- Output modification
- Compression via altered answers
- Forward prediction logic

Strict determinism is required.

4.6 Dataset Neutrality Requirement

Conformance must not depend on:

- Specific datasets
- Specific industries
- Specific hardware
- Specific numeric ranges

Core conformance must be demonstrable using deterministic operators.

External datasets may demonstrate applicability — but they do not define conformance.

Structural invariants define conformance.

4.7 Deterministic Environment Discipline

Implementations must explicitly control and declare all environment-sensitive execution factors.

The following must be strictly deterministic:

- File ordering (stable and explicitly defined)
- Encoding format (declared and fixed; platform-independent)
- Numeric formatting precision
- Floating-point arithmetic precision and rounding rules
- Hash algorithm (fixed and declared)
- Line termination discipline (explicitly normalized)
- Byte serialization format (fully specified and platform-independent)
- Artifact generation ordering

Floating-point arithmetic must be explicitly normalized to deterministic precision rules. All rounding behavior must be fixed, declared, and replay-stable.

Execution must not depend on:

- System clock
- Thread scheduling
- Random seed
- Unordered container iteration
- Parallel nondeterministic reduction
- OS-specific filesystem ordering
- Locale-dependent formatting
- Hardware-dependent floating behavior (unless explicitly normalized)

If identical declared inputs executed under different environments produce artifact differences, conformance fails.

Environmental nondeterminism invalidates replay identity.

Final authority condition:

```
B_A = B_B
```

Strict environmental determinism is mandatory for conformance.

4.8 Structural Freeze Boundary Requirement

A certified SSC-Core release must define a deterministic freeze boundary including:

- Core scripts
- Verification utilities
- Reference outputs
- Structural manifests

Excluded from freeze boundary:

- Documentation
- Diagrams
- Presentation materials
- Visualization tools

Modification inside the freeze boundary invalidates certification.

Presentation materials may evolve independently.

4.9 Multi-Stage Verification Requirement

Civilization-grade demonstration must include:

Phase 1 — Deterministic primary execution
Phase 2 — Deterministic replay execution
Phase 3 — Replay comparison
Phase 4 — Trace-derived recomputation
Phase 5 — Tamper ritual validation

All phases must produce PASS.

If any phase fails, conformance fails.

4.10 Final Structural Condition

SSC-Core conforms if and only if it preserves:

- Conservative collapse invariant
  `phi((m,a,s)) = m`
- Deterministic work model
- Deterministic structural field `Psi(t)`
- Deterministic compression `Omega`
- Deterministic curvature `K`
- Deterministic fingerprint `F`

- Replay identity

```
B_A = B_B
```

Structural conformance is computational — not interpretive.

---

5. Structural Applications, Boundaries & Non-Claims

5.1 What SSC-Core Does

SSC-Core computes **deterministic structural work geometry** alongside classical execution.

It produces:

- Structural work field `Psi(t)`
- Compression integral `Omega`
- Structural curvature `K`
- Structural density `D`
- Deterministic topology fingerprint `F`
- Replay-verifiable certificate

SSC-Core makes execution topology measurable.

It reveals:

- How work was distributed
- How compression evolved
- Whether execution oscillated structurally
- Whether two executions are geometrically identical

SSC-Core adds a **structural measurement axis** to computation.

It does not replace classical evaluation.

5.2 What SSC-Core Does Not Do

SSC-Core does **not**:

- Predict performance outcomes
- Guarantee optimization benefits
- Replace algorithm analysis
- Evaluate asymptotic complexity
- Compute entropy
- Estimate probability
- Test randomness quality

- Validate cryptographic strength
- Prove correctness
- Provide safety certification
- Replace domain mathematics

SSC-Core measures geometry of execution — nothing more.

5.3 Structural Applicability Condition

SSC-Core applies to any system that provides:

• deterministic ordered execution
• declared work model `W(t)`
• deterministic allow rule `W_allow(t)`
• deterministic decision stream `g(i)`
• replay-stable artifact generation

If these conditions are satisfied, SSC-Core can measure **structural execution geometry**.

SSC-Core instrumentation observes execution without modifying the classical operator.

Existing deterministic algorithms can therefore be instrumented to emit SSC-Core execution traces.

A compliant implementation exposes deterministic step records containing:

```
t, W(t), W_allow(t), Psi(t), g(t)
```

These traces allow structural quantities such as `Psi(t)`, `Omega`, `K`, and `F` to be derived directly from execution evidence.

Certificates must always be **trace-derived** and **replay-verifiable**.

Replay authority is defined by:

```
B_A = B_B
```

SSC-Core is **domain-neutral**.

It applies to:

• arithmetic algorithms
• data pipelines
• control logic
• deterministic AI inference passes
• compilers
• embedded systems
• simulation engines

Applicability depends on determinism.

Non-deterministic systems must first enforce deterministic discipline before SSC-Core instrumentation can be applied.

5.4 Non-Claims of Performance Superiority

SSC-Core does not claim:

- Faster execution
- Better compression
- Superior topology
- Improved efficiency
- Algorithmic advantage

It measures structure.

Performance benefits, if any, are separate from structural measurement.

Structural compression is not equivalent to speed improvement.

Measurement is not optimization.

5.5 Structural Divergence Awareness

SSC-Core enables detection of:

- Structural divergence under identical outputs
- Hidden compression instability
- Execution turbulence
- Replay instability

However, detection does not imply defect.

SSC-Core reports structure — it does not interpret it.

Interpretation remains external to the standard.

5.6 Separation from Governance Systems

SSC-Core is not:

- A runtime policy engine

- A governance enforcement layer
- A regulatory compliance system
- A blockchain-like authority mechanism

It is a **deterministic computation measurement framework**.

Certificate enforcement proves identity — not governance.

5.7 Research & Engineering Boundary

SSC-Core is suitable for:

- Structural benchmarking research
- Deterministic performance geometry analysis
- Execution topology comparison
- Academic reproducibility studies
- Deterministic AI inference trace analysis
- Infrastructure integrity verification

SSC-Core is not suitable for:

- Autonomous safety decisions
- Real-time fault intervention
- Probabilistic modeling environments
- Systems requiring stochastic exploration

SSC-Core assumes deterministic discipline.

5.8 Independence from Magnitude Semantics

Structural geometry is independent of output meaning.

Two outputs with different semantic interpretation may have identical structural geometry.

Two outputs with identical values may have different structural geometry.

SSC-Core measures structure — not meaning.

5.9 Civilization-Grade Positioning

SSC-Core becomes civilization-grade only when:

- Structural quantities are trace-derivable

- Replay identity is enforced
- Tamper ritual is validated
- Freeze boundary is declared
- Determinism is absolute

Civilization-grade status is not declared.

It is demonstrated.

Authority condition:

```
B_A = B_B
```

5.10 Final Declaration

SSC-Core produces:

- Classical truth `m_out`
- Structural work geometry `Psi(t)`
- Compression measure `Omega`
- Curvature measure `K`
- Structural density `D`
- Deterministic topology fingerprint `F`
- Replay-verifiable certificate

SSC-Core defines an additional deterministic computation layer that operates alongside classical execution.

It preserves magnitude exactly:

```
phi((m,a,s)) = m
```

It enforces execution identity strictly:

```
B_A = B_B
```

All structural quantities are:

- Deterministic
- Trace-derivable
- Replay-stable
- Byte-identical under identical declared inputs

SSC-Core introduces structural geometry as a formally measurable axis of deterministic computation.

Structural conformance is computational — not interpretive.

Appendix A — Mathematical Foundations of Structural Work Geometry

A.1 Deterministic Execution Model

Let a deterministic operator `O` execute over discrete steps `i = 1..N`.

At each step:

- Classical work increment: `w_i > 0`
- Decision stream: `g(i) in {0,1}`

Cumulative classical work:

```
W(t) = sum_{i=1..t} w_i
```

Total classical work:

```
W = W(N)
```

Cumulative allowed work:

```
W_allow(t) = sum_{i=1..t} g(i) * w_i
```

Constraints

- `0 <= W_allow(t) <= W(t)`
- `w_i > 0` for all `i`
- `W(t)` must be strictly increasing:
  `W(t) > W(t-1)` for all `t >= 1`
- `w_i` must be deterministically defined
- `g(i)` must be deterministically defined

Strict monotonicity of `W(t)` ensures:

- Denominator stability for `Psi(t)`
- Absence of zero-work steps
- Well-defined structural field computation

No zero-increment work steps are permitted under the declared work model.

The execution model must be fully reproducible from declared inputs and operator identity.

Determinism at the level of `w_i` and `g(i)` is mandatory for structural conformance.

A.2 Structural Work Field

The structural work field is defined as:

```
Psi(t) = W_allow(t) / W(t)
```

Domain:

```
0 < Psi(t) <= 1
```

Boundary conditions:

- If `g(i) = 1` for all `i`, then `Psi(t) = 1`
- If `g(i) = 0` for some `i`, then `Psi(t) < 1` for `t >= i`

Properties:

1. Deterministic
2. Trace-derivable
3. Replay-stable
4. Independent of output magnitude

A.3 Discrete Structural Compression

Discrete structural compression:

```
Omega = sum_{t=1..N} (1 - Psi(t))
```

Alternate form:

```
Omega = sum_{t=1..N} (W(t) - W_allow(t)) / W(t)
```

Lower bound:

```
Omega >= 0
```

Collapse condition:

If `g(i) = 1` for all `i`, then:

```
Omega = 0
```

Upper bound:

```
Omega <= N - sum_{t=1..N} (W_allow(t)/W(t))
```

`Omega` measures cumulative deviation from classical work.

A.4 Continuous-Time Analog

If execution admits continuous representation:

Let:

`W(t)` be differentiable
`W_allow(t)` be differentiable

Then:

```
Psi(t) = W_allow(t) / W(t)
```

Structural compression integral:

```
Omega = integral_0^T (1 - Psi(t)) dt
```

This treats structural compression as geometric area under deviation curve.

The discrete model is primary.

Continuous form is interpretive convenience.

A.5 Structural Curvature

Decision stream:

```
g(i) in {0,1}
```

Structural curvature:

```
K = sum_{i=2..N} |g(i) - g(i-1)|
```

Equivalent interpretation:

```
K = number of transitions between 0 and 1
```

Bounds:

```
0 <= K <= N-1
```

Collapse condition:

If `g(i) = 1` for all `i`:

```
K = 0
```

Maximum oscillation:

If `g(i)` alternates each step:

```
K = N-1
```

`K` measures topological switching complexity.

A.6 Structural Density

Structural density:

```
D = W_allow(N) / W(N)
```

Properties:

- `0 < D <= 1`
- `D = 1` under collapse mode
- Independent of curvature
- Independent of compression integral magnitude

Density measures endpoint compression.

A.7 Structural Geometry Vector

Define structural geometry vector:

```
G = (Omega, K, D, F)
```

Two executions are structurally equivalent if:

```
G_A = G_B
```

Output equivalence alone does not imply structural equivalence.

A.8 Structural Identity Theorem

Theorem (Replay Identity Necessity):

If two executions produce identical artifacts and identical manifests, then:

```
Omega_A = Omega_B
K_A = K_B
D_A = D_B
F_A = F_B
```

Proof:

Structural quantities are deterministically derived from `ssc_trace.csv`.

Manifest equality implies byte-identical trace.

Therefore, recomputation yields identical structural quantities.

Thus:

`B_A = B_B` implies `G_A = G_B`

A.9 Collapse Baseline Theorem

If `g(i) = 1` for all `i`, then:

- `W_allow(t) = W(t)`
- `Psi(t) = 1`
- `Omega = 0`
- `K = 0`
- `D = 1`

Proof:

Direct substitution into definitions.

Collapse mode establishes structural baseline.

A.10 Structural Divergence Condition

Structural divergence occurs when:

`m_out_A = m_out_B`
but
`G_A != G_B`

This implies:

Execution topology differs despite output equality.

Therefore, classical output comparison is insufficient for structural equivalence.

A.11 Deterministic Fingerprint Canonicalization

Fingerprint construction:

```
F = hash(g_stream || "|" || Psi_samples || "|" || Omega || "|" || K)
```

Where:

- `g_stream` = full deterministic decision sequence
- `Psi_samples` = fixed deterministic sampling schedule
- `Omega` = structural compression integral
- `K` = structural curvature

Canonicalization Requirements

The fingerprint construction must satisfy:

- Fixed concatenation order
- Fixed sampling schedule
- Fixed numeric formatting rules
- Fixed encoding format (explicitly declared)
- Platform-independent byte serialization
- Fixed hash algorithm (declared in OIR)
- Deterministic line termination discipline

The exact byte-level serialization format must be fully specified and independent of operating system, locale, hardware, or runtime implementation.

No implicit casting, locale-dependent formatting, or platform-specific floating representation is permitted.

All numeric quantities included in the fingerprint must be serialized using explicitly declared deterministic precision rules.

Structural Properties of F

`F` must be:

- Deterministic
- Order-stable
- Replay-stable
- Trace-derivable
- Sensitive to any structural mutation

Any change to:

- `g_stream`

- `Psi_samples`
- `Omega`
- `K`
- Sampling rules
- Serialization rules

must produce a different fingerprint.

Fingerprint equality implies structural identity under declared canonicalization discipline.

Replay authority condition remains:

```
B_A = B_B
```

A.12 Determinism Constraint

All structural quantities must satisfy:

If inputs identical and configuration identical, then:

```
Psi_A(t) = Psi_B(t)
Omega_A = Omega_B
K_A = K_B
F_A = F_B
B_A = B_B
```

No stochastic process permitted in structural computation.

A.13 Independence from Output Magnitude

Structural geometry does not depend on numeric magnitude of `m_out`.

Changing output value while preserving execution structure does not alter:

- `Psi(t)`
- `Omega`
- `K`
- `D`

Structural geometry measures execution path, not value semantics.

A.14 Completeness Statement

The structural quantities:

- `Psi(t)`
- `Omega`
- `K`
- `D`
- `F`

are sufficient to characterize deterministic execution topology under SSC-Core discipline.

Together with replay identity:

```
B_A = B_B
```

they define structural identity of execution.

OMP